DEPLOYOPS

# Plan de mejora para un portfolio 10/10

*Diagnóstico de la versión 0.0.1, prioridades técnicas y roadmap hacia un MVP convincente*

**Objetivo del documento**

Definir qué mejorar, en qué orden y con qué criterios de aceptación para que DeployOps sea un proyecto de portfolio coherente, seguro, demostrable y fácil de ejecutar, sin sobreingeniería.

## BASE DE LA REVISIÓN

Repositorio analizado: deploy-ops-main.zip compartido en la conversación.

Archivos considerados: README.md, docs/ARCHITECTURE.md, docs/DOMAIN_AND_RULES.md, docs/DECISIONS.md, backend, frontend, Docker Compose y GitHub Actions.

Fecha del informe: 20 de octubre de 2026.

## PRINCIPIO RECTOR

**No se busca que DeployOps tenga más tecnologías que otros proyectos. Se busca que las funciones principales sean correctas, estén probadas y se puedan demostrar de punta a punta.**

# 1. Resumen ejecutivo

La base de DeployOps es prometedora para una versión 0.0.1. El repositorio ya define un problema concreto, una arquitectura en capas, reglas de estado, decisiones técnicas (ADR), un backend Spring Boot, persistencia PostgreSQL, un entorno Docker para la base de datos y un workflow inicial de CI. La meta no es reescribirlo: es cerrar las brechas entre la intención documentada y el comportamiento real.

| Dimensión | Evaluación orientativa | Interpretación |
|---|---|---|
| **Documentación y diseño** | 8/10 | Hay visión del producto, arquitectura, dominio y decisiones. Falta reflejar con más precisión qué está implementado hoy. |
| **Base técnica del backend** | 7,5/10 | Capas, DTOs, validación, transacciones, timeouts y tests de la máquina de estados son una base razonable. |
| **MVP integrado hoy** | En construcción | El frontend sigue prácticamente en la plantilla inicial; persistencia de checks, WebSockets y autenticación no están completos. |
| **Potencial de portfolio** | Alto | El dominio permite demostrar lógica de negocio, métricas, seguridad, tests y una integración de punta a punta. |

## Fortalezas que conviene conservar

- Arquitectura monolítica por capas: adecuada para el alcance inicial; no hace falta dividir en microservicios.
- Máquina de estados con tolerancia a fallos: HEALTHY, DEGRADED, DOWN y RECOVERING ya tienen tests relevantes.
- DTOs, Bean Validation, transacciones y timeouts HTTP: son decisiones útiles que vale la pena ampliar, no reemplazar.
- Documentación modular en README y docs/ con arquitectura, dominio y ADRs.
- Existe una primera base de automatización en GitHub Actions y un Dockerfile multi-stage para el backend.

## Brechas de mayor impacto

- Seguridad: la API no tiene autenticación/autorización y acepta URLs que el servidor luego consulta; si se publica, existe riesgo de SSRF.
- Precisión: la disponibilidad inicial se presenta como 100 % antes de comprobar el servicio; el cálculo de uptime omite incidentes que empezaron antes de la ventana.
- Historial: se mide la latencia, pero no se persiste un registro temporal por cada health check.
- Integración: hay decisiones y dependencias para WebSockets, email y reportes, pero todavía no se ve el flujo funcional completo.
- Evidencia: la CI existente se concentra en backend; falta cobertura de integración y un flujo frontend verificable.

> **La definición de “10/10” en este plan**
>
> Un 10/10 de portfolio no significa competir con una plataforma comercial de monitoreo ni garantizar madurez empresarial. Significa que la aplicación tiene alcance acotado, comportamiento coherente, seguridad razonable, pruebas automatizadas, demo reproducible y documentación honesta.

# 2. Diagnóstico del repositorio y de la documentación

La documentación actual es una ventaja, pero en algunos puntos describe el destino del proyecto como si ya fuera una capacidad disponible. Conviene mantener los documentos de diseño y agregar una separación visible entre “implementado”, “en desarrollo” y “planificado”.

| Área | Lo que muestra el ZIP | Qué ajustar |
|---|---|---|
| **Backend actual** | EndpointController, EndpointManagerService, HttpMonitorService, MetricsService, MonitoringScheduler; ServiceEntity e IncidentEntity. | Nombrar explícitamente el alcance actual: CRUD básico, checks HTTP secuenciales, estado e incidentes. |
| **Modelo de dominio** | DOMAIN_AND_RULES.md describe USER, ENVIRONMENT, HEALTH_CHECK_CONFIG y CHECK_RESULT. En el código solo se encuentran ServiceEntity e IncidentEntity. | Marcar esas entidades como diseño futuro o implementarlas cuando el roadmap llegue a esa fase. |
| **Tiempo real** | README y ARCHITECTURE describen WebSockets y eventos de estado. Hay dependencia websocket, pero no encontré la configuración ni el flujo de emisión/suscripción. | Cambiar la etiqueta a planificado hasta que exista una demo funcional con reconexión. |
| **Email, PDF y móvil** | Se mencionan JavaMailSender, reportes PDF y cliente móvil como características, mientras que el roadmap los deja sin completar. | Moverlos a “Roadmap” y postergarlos hasta cerrar el MVP web. |
| **Stack frontend** | README declara TailwindCSS y Vitest; package.json revisado no declara Tailwind y configura Jasmine/Karma. | Actualizar las tecnologías documentadas o incorporarlas deliberadamente. |
| **Entorno local** | docker-compose.yml levanta PostgreSQL, no toda la aplicación; start-dev.bat automatiza solo Windows. | Documentar comandos manuales para Windows/macOS/Linux; decidir más adelante si Compose levanta todo el stack. |

## Ajuste recomendado al README

- Arriba: descripción de una frase, estado actual, instrucciones para ejecutar y una captura real cuando exista UI.
- Secciones claras: Implementado hoy, En desarrollo, Roadmap, Arquitectura, Pruebas y Seguridad.
- Mantener badges y stack sincronizados con package.json y pom.xml; evitar presentar una dependencia instalada como una función terminada.
- Agregar “Limitaciones conocidas” durante la etapa inicial: sin autenticación, historial de checks pendiente, frontend en desarrollo, demo solo local.

> **Regla documental**
>
> Cada afirmación sobre una capacidad implementada debería ser comprobable en código, tests o una demo. La visión futura se conserva, pero se rotula como futura.

# 3. Prioridad P0: corregir antes de una demo pública

Estas tareas son de corrección y seguridad. No exigen añadir grandes funcionalidades y tienen un impacto desproporcionadamente alto en la calidad percibida y en el riesgo de la aplicación.

**P0**

### Proteger el monitor HTTP contra SSRF

EndpointCreateDTO valida el formato general de URL, pero HttpMonitorService usa esa URL para hacer una petición desde el backend. Antes de permitir registros de usuarios o exposición pública, definir una política de destinos HTTP/HTTPS; bloquear loopback, redes privadas, link-local y endpoints de metadatos; controlar resolución DNS y redirecciones; limitar frecuencia y cantidad de monitores. Una validación por regex o CORS no basta. Para la demo, preferir endpoints de prueba controlados.

**P0**

### No exponer detalles internos en errores

GlobalExceptionHandler devuelve ex.getMessage() en respuestas 500. Devolver un mensaje genérico y un identificador de correlación; conservar el stacktrace únicamente en logs del servidor. Los errores de validación pueden seguir devolviendo un 400 con mensajes claros y seguros.

**P0**

### Definir semántica de borrado

deleteEndpoint invoca deleteById sin comprobar explícitamente la existencia ni definir qué ocurre con incidentes asociados. Decidir entre baja lógica, conservar historial o cascada intencional; cubrir el caso inexistente con un contrato HTTP claro y asegurar que no queden datos huérfanos.

**P0**

### No declarar HEALTHY/100 % antes del primer check

createEndpoint inicializa el estado como HEALTHY y el controller responde uptime 100.0. Incorporar UNKNOWN o PENDING y expresar que la disponibilidad aún no se conoce hasta ejecutar mediciones suficientes.

**P0**

### Establecer límites de uso y configuración

Limitar número de monitores por usuario, longitud y formato de URL, intervalos mínimos de check y timeouts. Evitar que un usuario pueda crear una carga ilimitada o programar comprobaciones excesivas.

**P0**

### Añadir identidad antes de habilitar usuarios

La API actual no define autenticación ni autorización. Antes de publicar creación/lectura/borrado de monitores, implementar login seguro y aislamiento por propietario en todas las consultas. CORS por sí solo no protege recursos.

### Puerta de publicación

No desplegar un endpoint público que acepte URLs arbitrarias para ser consultadas por el servidor hasta implementar defensas SSRF. Para el portfolio, un entorno demo con servicios de prueba controlados es una alternativa más segura y más reproducible.

# 4. Prioridad P1: que las métricas y el dominio sean confiables

## 4.1 Persistir cada comprobación

Hoy HttpMonitorService obtiene código y latencia y MonitoringScheduler los registra principalmente en logs. Sin una entidad de resultado no existe una serie histórica fiable para graficar ni auditar. Incorporar CheckResultEntity (o nombre equivalente) vinculada al servicio.

| Campo sugerido | Uso |
|---|---|
| **id, service_id** | Identidad de la medición y relación con el endpoint. |
| **checked_at (UTC)** | Momento de la comprobación; preferir Instant o una estrategia UTC explícita. |
| **success / outcome** | Resultado normalizado, separado del texto de error. |
| **http_status** | Código HTTP cuando existe; nulo o un valor definido cuando falla antes de responder. |
| **latency_ms** | Tiempo de respuesta medido de forma monotónica (System.nanoTime para medir duración). |
| **error_category** | Timeout, DNS/conectividad, status no esperado, etc.; no persistir secretos ni mensajes arbitrarios. |

Añadir un índice por service_id + checked_at. Para un MVP no hace falta particionar tablas ni introducir una base de datos de series temporales. Definir una política de retención más adelante, después de conocer el volumen real.

## 4.2 Corregir cálculo de uptime

- Consultar incidentes que se solapan con toda la ventana [inicio, ahora], no solo incidentes cuyo startedAt cae dentro de ella.
- Para cada incidente, recortar el intervalo al rango medido: inicio efectivo = max(inicio de incidente, inicio de ventana); fin efectivo = min(resolución o ahora, fin de ventana).
- Evitar doble contabilización si existieran intervalos superpuestos; sumar intervalos de downtime ya fusionados.
- No usar 100 % como sustituto de “sin datos suficientes”. Distinguir uptime calculable, UNKNOWN y ausencia de checks.
- Definir precisión y redondeo: Duration.toMinutes() pierde fracciones de minuto. Probar intervalos de segundos, límites exactos y servicios recién creados.

## 4.3 Alinear reglas de check con el modelo

- La documentación modela expected_status; la implementación actual considera exitoso cualquier 2xx. Decidir si el éxito es “cualquier 2xx” o un código configurable y documentarlo.
- Mantener el estado del servicio y el historial de resultados como conceptos distintos: el estado es el resumen actual; cada check es una observación inmutable.
- Conservar las reglas actuales de 3 fallos consecutivos para DOWN y 2 éxitos para cerrar incidente, salvo que los tests y una decisión documentada justifiquen cambiarlas.
- Usar timestamps UTC coherentes en checks, incidentes y métricas.

> **Criterio de aceptación**
>
> Dado un servicio caído antes de que empiece la ventana de 30 días, el uptime debe reflejar la caída durante toda la porción de tiempo que intersecta la ventana. Los tests deben verificarlo con fechas controladas, sin depender del reloj real.

# 5. Prioridad P1: robustecer la API y el ciclo de monitoreo

## API y contratos

- Definir errores consistentes: 400 para entrada inválida, 401 para falta de autenticación, 403 para falta de permiso, 404 para recurso inexistente y 409 cuando exista un conflicto real.
- Mantener DTOs de request/response; no serializar entidades JPA directamente como contrato público.
- Añadir paginación y filtros para listas e historial cuando el volumen crezca; evitar devolver todo el historial sin límites.
- Agregar OpenAPI/Swagger para documentar rutas, parámetros, ejemplos y errores.
- Validar el cuerpo de respuesta que se descarga en health checks: en la mayoría de casos solo se necesita el estado y parte de los headers; evitar consumir cuerpos arbitrariamente grandes.

## Scheduler y HTTP

- Los checks actuales son secuenciales y usan fixedDelay de 60 segundos. Es aceptable en v0.0.1; medir cuánto tarda un ciclo antes de añadir concurrencia.
- Si la cantidad de endpoints lo exige, usar un pool acotado, timeouts y límites de concurrencia por destino. No paralelizar sin límites.
- Hacer inyectable/configurable el cliente HTTP en vez de construirlo directamente dentro del servicio; facilita tests y políticas de seguridad centralizadas.
- Usar System.nanoTime para medir duración; currentTimeMillis puede saltar si cambia el reloj del sistema.
- Registrar errores con contexto útil (serviceId, categoría, duración), evitando incluir credenciales o datos sensibles en logs.
- Asegurar que dos ejecuciones no puedan abrir dos incidentes activos para el mismo servicio. Si más adelante hay varias instancias de la app, revisar coordinación/locking del scheduler.

## Persistencia y evolución de esquema

- Hibernate ddl-auto=update puede servir para prototipar localmente. Antes de una demo desplegada, adoptar migraciones versionadas con Flyway o Liquibase.
- Documentar la política de borrado de incidentes y poner restricciones/índices coherentes en base de datos.
- Si se incorpora multi-tenancy, todas las búsquedas y mutaciones deben quedar limitadas al owner autenticado; comprobarlo también con tests negativos.

# 6. Prioridad P1: pruebas, CI y calidad comprobable

Actualmente hay tests útiles para la máquina de estados y un test de contexto. La siguiente mejora no es perseguir cobertura por porcentaje: es probar los límites que podrían producir métricas falsas, incidentes duplicados o exposición de datos.

| Tipo de prueba | Escenarios mínimos recomendados |
|---|---|
| **Unitarias: estados** | HEALTHY → DEGRADED; 3 fallos → DOWN y un incidente; éxito durante DEGRADED; DOWN → RECOVERING; recaída; 2 éxitos cierran el incidente; no duplicar incidente. |
| **Unitarias: métricas** | Incidente que cruza el comienzo de ventana; incidente abierto; servicio recién creado; ventana sin checks; intervalos superpuestos; redondeo/precisión. |
| **HTTP monitor** | 2xx, 4xx/5xx, timeout, DNS/conexión fallida, redirección bloqueada y respuesta grande o lenta. |
| **Controller / API** | 201 al crear, validación 400, 404 al buscar recurso ausente, 401/403, usuario A no puede leer/borrar recurso de B. |
| **Integración PostgreSQL** | Persistencia real de endpoints, incidentes y resultados; claves foráneas y estrategia de borrado. |
| **Frontend** | Estados de carga, error y lista vacía; creación y borrado; reconexión de eventos y actualización de métricas. |

## Mejoras de CI

- Mantener el workflow backend y verificar que corre en push y pull request de las ramas principales; ejecutar Maven con el objetivo de verificación apropiado.
- Añadir un workflow de frontend que instale dependencias con lockfile y ejecute build y tests. El README debe coincidir con el runner real (hoy package.json usa Jasmine/Karma, no Vitest).
- Preferir PostgreSQL de prueba reproducible. Testcontainers es una opción para tests de integración independientes de una base local preconfigurada.
- Agregar un badge de CI solo cuando el workflow esté realmente pasando; no publicar badges de cobertura sin reporte automático real.
- Opcional, una vez estable: análisis estático/lint, auditoría básica de dependencias y escaneo de imagen/container.

> **Criterio de aceptación**
>
> Una pull request que rompa la máquina de estados, el cálculo de uptime o el build Angular debe quedar roja automáticamente. No hace falta desplegar automáticamente cada cambio a producción.

# 7. Prioridad P2: cerrar la experiencia de producto

El frontend no necesita ser enorme. Necesita representar con claridad lo que el backend sabe, incluir los flujos principales y recuperarse con elegancia de errores de red. Primero debe funcionar con REST; WebSockets se añaden después como mejora de experiencia.

## Pantallas mínimas del dashboard

| Vista | Contenido y comportamiento esperado |
|---|---|
| **Resumen** | Totales de servicios HEALTHY / DEGRADED / DOWN / UNKNOWN; última actualización; mensajes claros si aún no existen datos. |
| **Lista de servicios** | Nombre, URL o host permitido, estado, última comprobación, latencia reciente y uptime calculado. Crear, abrir detalle y borrar con confirmación. |
| **Detalle de servicio** | Gráfico de latencia a partir de CheckResult, porcentaje de uptime con ventana declarada y lista de incidentes del servicio. |
| **Incidentes** | Abiertos primero, duración, inicio, resolución, estado; filtros y paginación si el historial crece. |
| **Formularios / estados** | Validación amigable, loading, errores recuperables, empty states, responsive y accesibilidad básica de teclado/contraste. |

## Integración WebSocket después de REST

- Al abrir el dashboard, cargar una fotografía inicial mediante REST; WebSocket no reemplaza la persistencia ni el endpoint de consulta.
- Definir un contrato de evento pequeño y versionable: serviceId, status, timestamp y datos mínimos del cambio.
- Emitir eventos después de confirmar la transacción de base de datos; no anunciar un estado que todavía podría revertirse.
- Implementar reconexión con backoff. Al reconectar, volver a sincronizar el estado mediante REST porque algún evento pudo perderse.
- Con autenticación activa, autorizar la suscripción a los tópicos; no enviar datos privados de todos los usuarios a todos los clientes.

## La demo diferenciadora

> **Escenario recomendado para el video del portfolio**
>
> Crear un pequeño servicio de prueba controlado por Docker Compose con rutas /ok, /slow y /fail o un modo “flaky”. Mostrar que DeployOps registra latencia, tolera un fallo aislado, abre un incidente después de tres fallos, actualiza el dashboard y cierra el incidente después de dos éxitos. Esto es reproducible, seguro y más convincente que depender de que una API pública externa esté caída justo durante la demo.

# 8. Prioridad P2: ejecución, despliegue y presentación profesional

## Entorno reproducible

- Crear un .env.example sin secretos reales. Mantener credenciales por defecto solo para desarrollo local y documentar que no deben usarse en producción.
- Mejorar docker-compose para que un nuevo colaborador pueda levantar la base y, cuando sea oportuno, backend y frontend con un único comando.
- Agregar healthchecks de contenedor y esperar a que PostgreSQL esté listo antes de arrancar el backend.
- Ofrecer instrucciones multiplataforma; start-dev.bat no cubre macOS/Linux. Documentar comandos separados para Java/Maven, Angular y Docker.
- Usar migraciones de base de datos para despliegues; no depender de ddl-auto=update como estrategia de producción.
- Ejecutar el contenedor con privilegios mínimos y mantener imágenes/dependencias actualizadas.

## Observabilidad mínima

- Logs estructurados o al menos consistentes, con serviceId, outcome y duración; nunca volcar secretos o URLs con credenciales.
- Actuator con endpoints expuestos de forma selectiva: health para liveness/readiness, sin publicar métricas sensibles o environment indiscriminadamente.
- Métricas internas útiles: duración del ciclo del scheduler, cantidad de checks, fallos del monitor y tiempo entre checks.
- Configurar límites de retención para el historial cuando se sepa cuánto crece; evitar prematuramente particiones o infraestructura adicional.

## README y presencia de portfolio

- Descripción de una frase y captura real del dashboard en la parte superior.
- Demo online o video corto de 30-60 segundos, con un flujo real y no solo capturas estáticas.
- Sección “Qué aprendí / decisiones técnicas”: por qué monolito, por qué esos umbrales, cómo se calcula uptime y cómo se protege el monitor HTTP.
- Diagrama de arquitectura y diagrama de estados que coincidan con la versión implementada.
- Pasos de instalación, variables de entorno, comandos de tests y ejemplos de requests/errores.
- Roadmap y limitaciones conocidas honestas; enlaces a OpenAPI y workflow de CI si están disponibles.

# 9. Roadmap propuesto: de v0.0.1 a un MVP de portfolio

Los números son orientativos. Importa el orden y que cada hito tenga un resultado observable; no es necesario crear una release para cada cambio pequeño.

| Versión / fase | Alcance | Criterio de salida |
|---|---|---|
| **v0.0.2 - Core confiable** | Estado UNKNOWN/PENDING; respuesta inicial sin uptime ficticio; cálculo de uptime corregido; política de borrado; errores 500 seguros; primera defensa SSRF. | Tests para bordes de métricas/estados; no se devuelve detalle de excepciones; contrato HTTP coherente. |
| **v0.0.3 - Historial** | CheckResult persistido con timestamp, latencia, código y outcome; consultas por ventana; índices básicos. | Una comprobación queda en DB y alimenta una vista histórica; los cálculos usan la ventana real. |
| **v0.0.4 - Seguridad y contrato** | Autenticación/autorización; ownership; validación fuerte de destino y límites de uso; OpenAPI; migraciones de esquema. | Pruebas demuestran que un usuario no accede a datos de otro; destinos internos bloqueados en casos de prueba. |
| **v0.0.5 - Dashboard** | Angular conecta con REST; resumen, lista, detalle, incidentes, estados de carga/error/empty; tests frontend. | Un usuario puede ejecutar los flujos principales sin editar la base manualmente. |
| **v0.1.0 - Portfolio MVP** | WebSockets con reconexión y sincronización REST; demo reproducible; CI backend/frontend; Docker Compose documentado; README, diagramas, capturas y video. | Una persona externa puede clonar, levantar, probar la caída controlada y ver su recuperación siguiendo la documentación. |

## Extras para después de v0.1.0

- Notificaciones por email: implementar solo después de tener incidentes persistidos y destinatarios autenticados; probar idempotencia y fallos SMTP.
- Reportes PDF semanales: incluir período, uptime, incidentes y método de cálculo; tests de timezone y ausencia de datos.
- Landing pública: usar solo datos agregados o servicios demo curados. No mostrar URLs privadas, propietarios ni datos de usuario.
- Aplicación móvil: postergar hasta que el producto web funcione. Antes de comprometerse con una tecnología, definir caso de uso real y validar el framework elegido.
- Escalabilidad: concurrencia limitada o coordinación de scheduler multi-instancia solo cuando haya evidencia de que el procesamiento secuencial es insuficiente.

# 10. Checklist de “portfolio 10/10”

Usá esta lista como puerta de calidad antes de presentar DeployOps como proyecto terminado. No todos los extras son obligatorios; sí lo son la honestidad, la reproducción de la demo y la confiabilidad del núcleo.

| Criterio | Evidencia para marcarlo como cumplido |
|---|---|
| **Dominio correcto** | Estados, incidentes y recuperación cumplen reglas documentadas y los tests las protegen. |
| **Métricas correctas** | Uptime valida ventanas, incidentes abiertos, minutos/segundos límite y falta de observaciones. |
| **Seguridad base** | Autenticación y ownership; SSRF mitigado; errores no filtran internals; límites razonables. |
| **Datos históricos** | Checks y latencias se almacenan y consultan por ventana con índices/paginación necesarios. |
| **Frontend conectado** | Dashboard útil, responsive, estados vacíos y de error; WebSocket reconecta y sincroniza. |
| **Tests y automatización** | CI ejecuta backend y frontend; pruebas de integración cubren PostgreSQL y casos críticos. |
| **Entorno reproducible** | Instrucciones claras y un camino rápido para arrancar la demo sin secretos ni pasos ocultos. |
| **Presentación** | README con capturas, diagrama actual, demo o video, comandos y decisiones técnicas defendibles. |
| **Documentación honesta** | Lo implementado se distingue de lo planificado; stack y diagramas reflejan el código actual. |
| **Alcance enfocado** | No hay tecnologías añadidas solo por moda; las decisiones se pueden justificar con necesidades reales. |

## Qué significa el 10 en este caso

Un proyecto no obtiene un 10 por tener 100 funcionalidades. Lo obtiene, dentro de la escala subjetiva de portfolio, cuando su alcance está bien elegido y cada afirmación está respaldada por código, tests, instrucciones o una demostración. Se puede reconocer que es un proyecto personal y de alcance limitado, y aun así mostrar disciplina de ingeniería.

> **Prioridad más importante**
>
> Si el tiempo es limitado, completá el flujo vertical: crear servicio → ejecutar check → persistir resultado → detectar caída → abrir incidente → mostrarlo → recuperar servicio → cerrar incidente. Un flujo pequeño, correcto y demostrable vale más que cinco features a medio terminar.

# 11. Referencias técnicas recomendadas

No hace falta leer toda la documentación antes de seguir programando. Usá estas referencias al implementar cada fase. Son fuentes oficiales o de referencia para las prácticas mencionadas.

**OWASP - Server Side Request Forgery Prevention Cheat Sheet**

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

Guía central para validación de destinos, redirecciones y mitigación de SSRF.

**Spring Boot - Testing**

https://docs.spring.io/spring-boot/reference/testing/index.html

Punto de partida para tests unitarios, slice tests e integración en Spring Boot.

**Testcontainers for Java**

https://java.testcontainers.org/

Para levantar PostgreSQL efímero en pruebas de integración reproducibles.

**GitHub Docs - Building and testing Java with Maven**

https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven

Referencia para ampliar el workflow de CI de backend.

**Docker Docs - Docker Compose**

https://docs.docker.com/compose/

Para documentar y reproducir el entorno local.

**OpenAPI Specification**

https://spec.openapis.org/oas/latest.html

Contrato estándar para describir la API HTTP.

**Spring Security Reference**

https://docs.spring.io/spring-security/reference/

Referencia para incorporar autenticación y autorización en Spring.

## Nota sobre el alcance de la revisión

Este plan se elaboró a partir del ZIP compartido: README, documentación de arquitectura/dominio/decisiones, fuentes del backend y tests, package.json, Docker Compose, Dockerfile y workflow de GitHub Actions. No afirma que el código haya pasado toda la suite en este entorno ni reemplaza una revisión de seguridad profesional. Las notas de calidad son orientativas y el orden propuesto puede ajustarse con cada iteración.